

Construindo um HSM de brinquedo

Como funciona um módulo criptográfico de hardware, aprendido de dentro para fora

Projeto hsm-a7 — Artix-7 XC7A35T

Sumário

Parte I — O que é um HSM

1. O problema que um HSM resolve
2. Modelo de ameaça
3. As três propriedades
4. Onde HSMs são usados

Parte II — Arquitetura interna

5. A fronteira criptográfica
6. Hierarquia de chaves
7. Cerimônia de chave, split knowledge e dual control
8. Key blocks: por que o formato é feio
9. Máquina de estados: a chave de switch
10. Aleatoriedade: onde HSM vira ciência
11. Self-test: por que o dispositivo se recusa a funcionar
12. Zeroização: apagar é mais difícil do que parece
13. Log de auditoria: por que antes, não depois
14. Ciclo de vida de uma chave

Parte III — Como HSMs são atacados

15. Ataques de API — o mais importante
16. Canais laterais
17. Injeção de falha
18. Ataques físicos e antitamper

Parte IV — Certificação

19. FIPS 140-3
20. PCI: as regras de pagamentos
21. Common Criteria
22. O que “certificado” significa — e o que não significa

Parte V — O projeto hsm-a7

23. Objetivo, plataforma e restrições
24. Fase 1, peça por peça
25. Resultados medidos
26. Três lições que a Fase 1 ensinou fazendo

Parte VI — O caminho adiante

27. Fase 2 — engines e self-test (*em andamento*)
28. Fase 3 — hierarquia de chaves
29. Fase 4 — armazenamento não-volátil
30. Fase 5 — API de host
31. Fase 6 — tamper e ataques
32. Fase 7 — assimétrico
33. Ordem de trabalho, e a regra que a sustenta

Apêndices

- A. Glossário
- B. Especificação do protocolo
- C. Pinagem
- D. Reproduzir o trabalho
- E. Leitura

Sobre este documento

Este é o material de um projeto didático: construir, do zero, um módulo criptográfico com hierarquia de chaves, cerimônia de carga, key blocks e API de host — reproduzindo a arquitetura de um HSM real em escala reduzida, num FPGA Artix-7.

O documento tem duas metades que se encontram no meio. As Partes I a IV explicam **como um HSM funciona e por quê**, independentemente deste projeto: arquitetura, ameaças, ataques históricos e certificação. A Parte V explica **o que foi construído** e liga cada peça de código ao conceito correspondente. A Parte VI é o roteiro do que falta.

Estado na data desta edição: **Fase 1 completa e validada em hardware; Fase 2 em andamento**, com os cores de AES-256 e SHA-256 verificados contra os vetores oficiais do NIST em simulação.

Quem quer só entender o assunto pode parar na Parte IV. Quem quer reproduzir o trabalho começa na Parte V e volta às anteriores conforme a dúvida aparecer.

AVISO — este dispositivo não é um HSM

Não há PUF, malha antitamper, sensores físicos, gerador de números aleatórios certificado, nem qualquer validação FIPS, PCI ou Common Criteria. As chaves ficam numa BRAM que qualquer um com o bitstream pode inspecionar em simulação.

O objetivo é **entender a arquitetura de dentro para fora**, não proteger material de chave real. Nenhuma chave de produção deve jamais ser carregada neste dispositivo.

A honestidade sobre isso não é modéstia: saber exatamente **por que** este aparelho não é seguro é metade do aprendizado. Cada ausência listada na seção 22 corresponde a um mecanismo real que existe num HSM de verdade e custa dinheiro, tempo de projeto e meses de avaliação.

Parte I — O que é um HSM

1. O problema que um HSM resolve

Software comum guarda chaves na memória do processo. Quem tem acesso à máquina — o usuário root, um dump de memória, um bug de leitura fora de limites, um backup mal apagado — tem a chave. E há uma consequência mais sutil: **não existe forma de distinguir “usar a chave” de “copiar a chave”**. Se o programa consegue assinar, o programa consegue exportar o segredo que assina. Quem controla o programa herda essa capacidade.

Um HSM (*Hardware Security Module*) existe para tornar essas duas coisas diferentes. Ele oferece uma **interface de operações**, não de acesso:

```
o host manda: "assine este bloco com a chave 7"
o host nunca manda: "me devolva a chave 7"
```

A chave nasce dentro do dispositivo, vive dentro e morre dentro. Ela pode sair — mas só embrulhada (*wrapped*) por outra chave que também nunca saiu.

Essa separação parece pequena e reorganiza tudo. A pergunta de segurança deixa de ser “*quem consegue ler a memória?*” e passa a ser:

O que a API permite deduzir sobre a chave, se for chamada um milhão de vezes com entradas escolhidas pelo atacante?

Praticamente toda a arquitetura descrita neste documento decorre dessa pergunta. É também por isso que os ataques mais interessantes contra HSMs não quebram criptografia: eles conversam com a API.

Um exemplo concreto

Imagine um banco que precisa verificar PINs de cartão. O PIN correto é derivado do número da conta com uma chave secreta. Sem HSM:

- a chave de derivação está na memória do servidor de autorização;
- qualquer comprometimento desse servidor entrega a chave;
- com a chave, o atacante calcula o PIN de **qualquer** conta, offline, sem deixar rastro e sem limite de tentativas.

Com HSM:

- a chave está dentro do módulo e nunca esteve em outro lugar;
- o servidor manda “este PIN confere para esta conta?” e recebe sim ou não;
- comprometer o servidor dá ao atacante a capacidade de **perguntar**, não a chave. Perguntas são contadas, limitadas e registradas.

O atacante não é impedido — ele é **reduzido a um canal estreito, medido e auditável**. Essa redução é o produto que um HSM vende.

2. Modelo de ameaça

“Seguro” não significa nada sem dizer contra quem. HSMs são projetados contra uma escada de adversários, e cada degrau custa mais caro de defender.

Adversário	Capacidade	Defesa correspondente
Aplicação comprometida	Chama a API à vontade	Máquina de estados, atributos de chave, limites de uso
Administrador do host	Root na máquina, vê todo o tráfego do canal	Chaves nunca no host; autorização física no próprio módulo
Insider com acesso físico	Toca no equipamento, opera o painel	Dual control, split knowledge, log de auditoria
Atacante com o aparelho na mão	Bancada, osciloscópio, fonte de alimentação	Antitamper, sensores, canal lateral, injeção de falha
Laboratório especializado	Decapsulação, microsonda, FIB	Malha ativa, camadas de blindagem, zeroização por detecção

Duas observações que orientam o resto do documento.

O host é hostil por premissa. Não é que se desconfie do host — é que a arquitetura não pode *depende* de ele estar íntegro. É por isso que autorização de operações sensíveis é feita com **botões no próprio equipamento**, e não com um comando “eu autorizo” que chega pelo cabo. Um comando pelo cabo tem exatamente a confiabilidade do host que o enviou.

A escada é econômica, não absoluta. Nenhum HSM é inviolável. O objetivo é tornar o ataque mais caro que o valor protegido, e detectável quando tentado. Um módulo que resiste seis meses a um laboratório nacional e falha contra ele no sétimo mês pode estar perfeitamente adequado, se as chaves tiverem rotação trimestral.

3. As três propriedades

Tudo que um HSM faz se organiza em três propriedades. Elas são interdependentes: qualquer uma sozinha é inútil.

Confinamento

Existe uma fronteira física, e material de chave em claro não a atravessa. Dentro: chaves, estado do gerador aleatório, segredos derivados. Fora: criptogramas, chaves embrulhadas, valores de verificação, identificadores, log.

Confinamento sem mediação é um cofre com a porta aberta.

Mediação

Toda operação passa por verificação de política **antes** de executar: o dispositivo está no estado certo? Há autorização suficiente? Esta chave tem permissão para esta operação? O pedido é bem formado?

Mediação sem confinamento é uma sugestão — se a chave pode ser lida por fora, a verificação é decorativa.

Evidência

O que aconteceu fica registrado de forma que sobreviva à falha e não possa ser reescrito para omitir a tentativa.

Sem evidência, você nunca descobre se as outras duas falharam. E é a propriedade mais frequentemente subestimada: um log que registra apenas sucessos mostra um dispositivo saudável exatamente enquanto ele está sendo mapeado por um atacante, cujo trabalho produz muitos erros e poucos acertos.

4. Onde HSMs são usados

O termo cobre famílias de equipamento bastante diferentes, e saber qual é qual ajuda a entender por que este projeto modela LMK e TR-31 em vez de, digamos, PKCS#11 e certificados.

HSM de pagamentos

O mais antigo e o mais rígido. Protege PINs de cartão, chaves de terminal e criptogramas EMV. Exemplos: Thales payShield, Utimaco Atalla.

É onde nasceram os conceitos de **LMK** (chave mestra local), **key block** e a exigência operacional de dual control e split knowledge. As regras não são recomendações: PCI PIN Security e PCI PTS HSM são requisitos contratuais para quem processa transações de cartão.

O comando típico não é “assine” — é “verifique este PIN”, “traduza este PIN block desta chave para aquela”, “gere uma chave de terminal”.

HSM de propósito geral

Protege chaves de PKI, assinatura de código, TLS, bancos de dados, blockchain. Exemplos: Entrust nShield, Thales Luna, Utimaco SecurityServer, YubiHSM.

A interface padrão é **PKCS#11** — `C_GenerateKey`, `C_Sign`, `C_WrapKey` — ou APIs próprias (JCE, CNG, KMIP). O modelo mental é o de um chaveiro com objetos que têm atributos, e os atributos é que definem o que se pode fazer com cada chave.

HSM em nuvem

AWS CloudHSM, Azure Managed HSM, Google Cloud HSM. Fisicamente são os mesmos módulos, operados pelo provedor, com o cliente controlando as chaves. O interesse conceitual está no problema novo que criam: **como você confia num módulo que não está na sua sala?** A resposta envolve atestação remota, e é uma área em movimento.

Elementos seguros e TPMs

Primos pequenos: TPM na placa-mãe, Secure Enclave, smartcard, YubiKey. Mesma ideia — chave que não sai, operação mediada — com orçamento de área, consumo e custo muito menores. Um cartão de crédito com chip é um HSM minúsculo cuja produção passa de bilhões de unidades por ano.

O que este projeto modela

O caminho de pagamentos: **LMK, key blocks TR-31, cerimônia de carga com componentes, dual control físico**. A razão é didática — é a família onde os mecanismos organizacionais (quem autoriza o quê, e como se prova) estão mais explícitos e mais bem documentados publicamente.

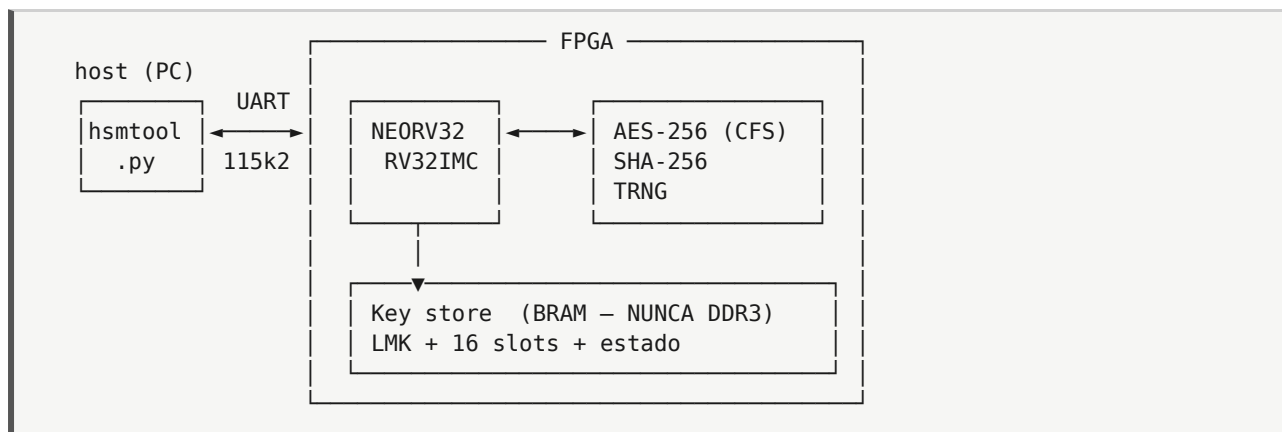
A Fase 5 do plano acrescenta um subconjunto de PKCS#11, para mostrar a outra interface sobre o mesmo núcleo.

Parte II — Arquitetura interna

5. A fronteira criptográfica

É o conceito central, e num projeto bem feito ele é literal e desenhável. Num documento de certificação FIPS existe um diagrama marcando o que está dentro, o que está fora, e uma lista de **todas** as portas que atravessam.

Neste projeto:



Dentro: LMK, chaves de trabalho em claro, estado do DRBG.

Fora: key blocks embrulhados, KCVs, criptogramas, *handles*, log.

Nada da primeira lista atravessa para a segunda. Todo comando novo é avaliado contra essa regra **antes** de ser implementado — é o único momento barato de perceber que um comando vaza.

Por que a fronteira precisa ser observável

Uma fronteira que você só pode *afirmar* não vale muito. Este projeto usa UART, e não Ethernet, precisamente por isso: a UART sai por dois pinos físicos identificáveis, e dá para prender um analisador lógico neles e capturar tudo que o dispositivo já disse.

O critério de aceitação da Fase 3 explora isso literalmente: rodar a suíte de testes inteira, capturar a UART, e fazer `grep` das chaves conhecidas do teste contra a captura. Se aparecer um byte, o projeto falhou — e você **sabe**, com evidência.

Com um stack de rede no meio, “nenhuma chave vazou” vira uma opinião sobre dezenas de milhares de linhas de código de terceiros. Com dois fios, vira uma medida.

Por que a memória importa mais do que parece

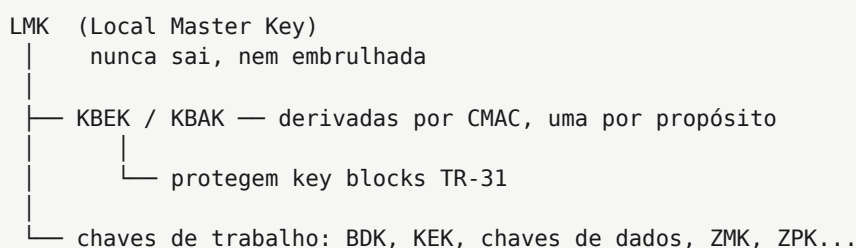
Neste projeto as chaves ficam em BRAM — memória interna ao die do FPGA. Lê-la exige decapsular o chip.

A DDR3 da placa é externa: o barramento passa por trilhas na PCB e chega a conectores de expansão. Uma ponta de prova resolve. Por isso a DDR3 não é apenas *evitada por convenção* — o barramento externo do processador está **desligado na configuração**, de modo que não existe caminho lógico do SoC até ela.

Essa é a diferença entre uma **política** e um **controle**: entre “o firmware não escreve chave na memória externa” e “não há circuito ligando o processador à memória externa”. A primeira depende de todo programador futuro; a segunda, não.

6. Hierarquia de chaves

Um HSM com dezesseis slots de chave não guarda dezesseis segredos. Guarda **um** — e protege ou deriva todos os outros a partir dele.



Três razões, e vale entender cada uma porque elas explicam decisões aparentemente arbitrárias mais adiante.

Backup e migração. Se cada chave fosse independente, mover o serviço para outro HSM significaria mover N segredos, cada um com seu próprio procedimento. Com a hierarquia, você move a LMK — por cerimônia, em componentes — e todos os key blocks existentes continuam válidos, porque eles sempre foram apenas dados cifrados sob ela.

Revogação em bloco. Zeroizar a LMK invalida instantaneamente **todo** key block protegido por ela, onde quer que esteja: em backup, em fita, no servidor de outra filial. É a única operação de destruição que se consegue executar em tempo constante sobre dados que não estão na sua posse.

Separação de propósito. KBK (chave de cifra) e KBAK (chave de autenticação) são derivadas da LMK por CMAC com rótulos distintos. A chave que cifra o corpo do key block nunca é a mesma que autentica seu cabeçalho. Se fosse, quebrar uma quebraria as duas — e existem ataques que exploram exatamente o reuso de chave entre modos de operação.

Atributos de chave

Cada chave carrega metadados que **fazem parte** dela, no sentido de que alterá-los invalida a autenticação. Estrutura usada neste projeto:

```
typedef struct {
    uint8_t in_use;
    uint8_t usage[2]; // TR-31: 'B0'=BDK, 'K0'=KEK, 'D0'=dados...
    uint8_t algorithm; // 'A'=AES, 'T'=3DES
    uint8_t mode_of_use; // 'E'=cifrar, 'D'=decifrar, 'B'=ambos, 'N'=nenhum
    uint8_t exportability; // 'E'=exportável, 'N'=não, 'S'=sensível
    uint8_t key_len;
    uint8_t key[32]; // BRAM. Nunca sai daqui em claro.
    uint8_t kcv[3];
    uint32_t use_count;
} key_slot_t;
```

`exportability` é o campo que mais ensina. Uma chave marcada `'N'` não pode sair do módulo **por caminho nenhum** — nem embrulhada. Se algum comando conseguir exportá-la, o comando está errado. É exatamente o tipo de propriedade que se testa exaustivamente, porque ela vale zero se houver um único caminho esquecido.

`use_count` existe porque uso é informação: uma chave que devia ser usada mil vezes por dia e foi usada um milhão indica alguém explorando a API.

7. Cerimônia de chave, split knowledge e dual control

A LMK não é gerada num lugar e copiada para outro. Ela é **montada dentro do dispositivo**, a partir de componentes que nenhuma pessoa vê juntos.

```
LMK = componente_A ⊗ componente_B ⊗ componente_C
```

Cada custodiante carrega apenas o seu componente. Nenhum componente isolado revela nada sobre a LMK — é o argumento do one-time pad: XOR com material desconhecido e uniforme não vaza informação. Isso é **split knowledge**.

KCV: verificar sem revelar

A cada passo, o dispositivo devolve um **KCV** (*Key Check Value*): os três bytes mais significativos do resultado de cifrar um bloco de zeros com a chave.

É um resumo pequeno o suficiente para não ajudar um atacante — 24 bits não permitem recuperar uma chave de 256 — e grande o suficiente para pegar digitação errada, componente trocado ou cartão corrompido. O host calcula o KCV esperado de forma independente e compara.

Repare no padrão: o dispositivo precisa provar que recebeu a coisa certa, sem revelar a coisa. Essa tensão reaparece o tempo todo em criptografia aplicada.

Dual control

Split knowledge diz que ninguém *sabe* a chave inteira. Dual control diz que ninguém *age* sozinho.

Neste projeto, `LMK_LOAD_COMPONENT` só é aceito com **dois botões físicos pressionados simultaneamente**. É simplório e é exatamente o ponto: separa fisicamente *quem digita* de *quem autoriza*. Um operador sozinho, mesmo com acesso administrativo total ao host, não carrega componente nenhum.

Por que isso existe, além de desconfiança: um operador que **não pode** agir sozinho não pode ser coagido com sucesso individual, não pode ser subornado individualmente, e não pode cometer o erro sozinho. Remove-se a possibilidade, não a tentação.

Num HSM comercial o mecanismo é mais rico — cada custodiante se autentica com smartcard e PIN, e o log registra *quem* autorizou. Mas a estrutura é a mesma, e a versão com dois botões torna visível o que a versão com smartcards esconde atrás de conveniência.

Por que a integridade do botão é código de segurança

Um contato mecânico produz dezenas de transições elétricas em poucos milissegundos ao ser pressionado ou solto. Sem filtragem, um único toque parece vários pressionamentos.

Se o firmware ler “pressionado” num ressalto que ocorreu **depois** de o custodiante soltar o botão, ele registrou uma autorização que ninguém deu. O mecanismo de autorização precisa refletir **intenção humana**, não estado elétrico de contato — e a distância entre as duas coisas é uns 10 ms de filtro.

8. Key blocks: por que o formato é feio

TR-31 (hoje ANSI X9.143) é o formato de troca de chaves da indústria de pagamentos. Um key block é ASCII, tem cabeçalho legível, e parece desnecessariamente complicado. Cada complicação existe por causa de um incidente.

```
D0112K0AB00N0000 <corpo cifrado em AES-CBC> <CMAC>
^^^^^^^^^^^^^^^^
cabeçalho, em claro, mas COBERTO PELO MAC
```

O cabeçalho declara o que a chave é: versão do formato, comprimento, **uso** (K0 = KEK, B0 = BDK, D0 = dados), algoritmo, **modo de uso** (só cifrar, só decifrar, ambos) e **exportabilidade**.

O ataque que o formato existe para impedir

Se o cabeçalho não fosse autenticado junto com o corpo, um atacante poderia pegar um key block legítimo e:

- trocar N (não exportável) por E (exportável), e depois exportar a chave em claro por um caminho autorizado;
- trocar o uso de “só verificar PIN” para “cifrar dados arbitrários”, e usar a chave de PIN como oráculo de cifra;
- trocar o algoritmo declarado, induzindo o módulo a usar a chave num modo para o qual ela não foi projetada.

A chave continuaria criptograficamente válida, o HSM a importaria sem reclamar, e ela passaria a fazer algo que o dono nunca autorizou.

Isso **não é hipotético**. É a razão histórica pela qual os formatos anteriores — os chamados *variant key blocks*, em que o tipo era codificado por XOR com um vetor de controle — foram abandonados pela indústria. Bond e Clulow mostraram famílias inteiras de ataques que consistiam em manipular esses vetores para converter uma chave de um tipo em outro.

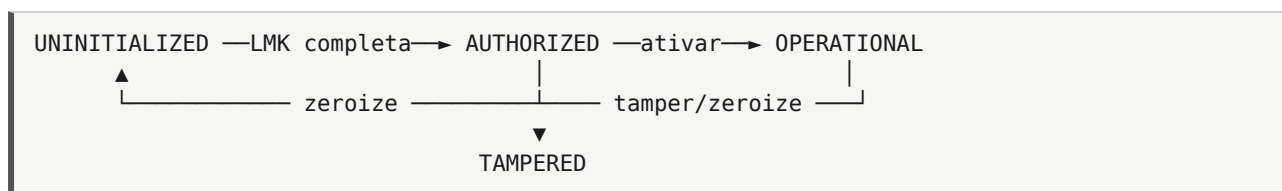
Consequência de projeto: **um byte errado no cabeçalho explode o MAC inteiro**. É o comportamento desejado, e é o que torna a implementação irritante de acertar.

Por que escrever o parser duas vezes

Neste projeto o TR-31 é implementado em C no firmware e em Python no host, e os dois validam-se mutuamente sobre blocos aleatórios.

É de longe a forma mais rápida de aprender o padrão de verdade, porque a menor divergência — um byte de preenchimento, uma ordem de campo, um comprimento incluído ou não na contagem — aparece imediatamente como MAC inválido. Não dá para “quase” implementar TR-31: ou os dois lados concordam em cada byte, ou nada funciona.

9. Máquina de estados: a chave de switch



HSMs comerciais têm uma chave física de três posições no painel. Este é o modelo.

Comandos sensíveis — carga de componente, exportação de material derivado da LMK, mudança de política — existem **apenas** em **AUTHORIZED**. E **AUTHORIZED** é um estado no qual o dispositivo **não atende produção** e no qual só se entra por ação física deliberada.

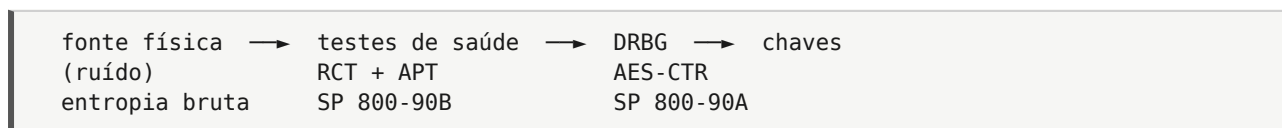
O raciocínio: em operação normal, a superfície de ataque deve ser mínima. Em **OPERATIONAL**, o dispositivo faz o trabalho e nada mais — não carrega chave, não exporta LMK, não muda política. Um invasor que comprometa **totalmente** o host de produção ainda não carrega componente de LMK, porque o dispositivo não aceita aquele opcode naquele estado, e mudar de estado exige mão humana no equipamento.

TAMPERED é absorvente: uma vez lá, só se sai por zeroize. E um dispositivo em **TAMPERED** deve responder o mínimo possível — o estado existe para proteger o que restou, não para continuar servindo.

10. Aleatoriedade: onde HSM vira ciência

Um HSM é tão bom quanto seu gerador. Chave previsível é chave pública, e o histórico de falhas nessa área é longo e caro.

A arquitetura padrão tem duas camadas:



Por que não usar a fonte física diretamente

Ruído físico tem viés e correlação. Osciladores em anel num FPGA — a fonte usada neste projeto — são sensíveis a temperatura e a tensão de alimentação. E um atacante que controle a fonte de alimentação pode **reduzir a entropia** sem que nada pareça errado do lado de fora.

O DRBG existe para transformar entropia imperfeita mas *suficiente* num fluxo computacionalmente indistinguível de aleatório. Ele não cria entropia — ele a estica e a uniformiza.

Testes de saúde: o coração da coisa

SP 800-90B exige que a fonte seja monitorada **continuamente**, não validada uma vez na fábrica. Dois testes obrigatórios:

RCT (*Repetition Count Test*) — dispara se a mesma amostra se repete acima de um limiar. Pega a falha catastrófica: o oscilador parou, a fonte travou num valor, o pino soltou.

APT (*Adaptive Proportion Test*) — janela de 512 ou 1024 amostras, dispara se um valor domina a janela. Pega a degradação: a fonte ainda oscila, mas ficou enviesada.

Falha em qualquer um → **TAMPERED**, DRBG parado, indicador vermelho. Não é paranoia: é a diferença entre “gerei uma chave fraca e não sei” e “me recusei a gerar”.

Escrever esses testes ensina mais sobre certificação de módulo criptográfico que qualquer texto sobre o assunto — eles são boa parte do motivo de uma avaliação levar meses.

11. Self-test: por que o dispositivo se recusa a funcionar

FIPS 140-3 exige que o módulo rode testes de resposta conhecida (**KAT**, *Known Answer Test*) na inicialização, **antes** de aceitar qualquer comando, e que entre em estado de erro se qualquer um falhar.

Parece burocracia. Não é, e o raciocínio é bonito:

Um AES que cifra errado não produz erro. Produz criptograma.

Você não descobre nada até tentar decifrar — possivelmente meses depois, possivelmente em outro dispositivo, possivelmente sobre o único backup que existia. Um bit preso numa memória, uma violação de temporização marginal que só aparece a 60 °C, uma corrupção na configuração do FPGA: todos se manifestam como criptografia silenciosamente errada.

Os KAT são o único momento em que o dispositivo consegue dizer **“eu não estou funcionando”** antes de causar dano.

Neste projeto os mesmos vetores rodam nos testbenches de simulação e no POST em hardware. Isso é deliberado e diagnóstico: se um KAT passa na simulação e falha no POST, o problema está no barramento ou na integração, não no núcleo criptográfico.

E há uma regra que decorre disso, adotada como inviolável neste projeto:

Se um KAT falha, o bug está no código, não no vetor. Não ajustar vetor, não relaxar assert, não marcar teste como ignorado.

Um teste enfraquecido para passar é pior que teste nenhum, porque agora existe uma afirmação falsa de correção, e alguém vai confiar nela.

12. Zeroização: apagar é mais difícil do que parece

```
memset(chave, 0, 32); /* ...e a função termina aqui */
```

Para o compilador, isso é um *dead store*: escrita num buffer que ninguém lê depois. O padrão C **permite** eliminá-lo, e com otimização ativada o compilador elimina. A chave permanece na memória, intacta, e o código *parece* correto na revisão — inclusive para um revisor experiente.

A solução é forçar o compilador a tratar cada escrita como efeito colateral observável:

```
void wipe(void *p, size_t n)
{
    volatile uint8_t *q = (volatile uint8_t *)p;
    while (n--) *q++ = 0u;
    __asm__ __volatile__("" ::: "memory");
}
```

`volatile` impede a eliminação; a barreira de memória impede reordenamento.

Zeroização é **requisito explícito** do FIPS 140-3, e a exigência vai além de chamar a função: um módulo certificado precisa demonstrar que o material foi efetivamente destruído. Por isso, neste projeto, o comando `ZEROIZE` sobrescreve a região com padrão e depois com zeros, e o **teste faz dump da região e verifica**. “Chamei a função de apagar” não é evidência.

13. Log de auditoria: por que antes, não depois

O registro é gravado **antes** de executar o comando. Isso é contraintuitivo — parece natural registrar o resultado.

O motivo: um log escrito depois só registra sucesso. Tentativa que falhou, comando que travou o dispositivo, sequência que causou reset — nada disso aparece. E é exatamente essa a informação que importa numa investigação.

Um atacante sondando a API produz **muitos erros e poucos sucessos**. Um log de sucessos mostra um dispositivo perfeitamente saudável enquanto ele está sendo mapeado.

Requisitos que decorrem:

- **contador monotônico**, para detectar remoção de entradas;
- **gravação antes da execução**, para capturar o que travou;
- **sem material de chave**, obviamente;

- **armazenamento que sobrevive à queda de energia**, senão o ataque termina com um puxão no cabo.

14. Ciclo de vida de uma chave

Amarrando as seções anteriores: uma chave, do nascimento à morte, e onde cada mecanismo entra.

Fase	O que acontece	Mecanismo
Geração	Dentro do módulo, a partir do DRBG	Testes de saúde da fonte, POST
Atribuição	Recebe atributos: uso, modo, exportabilidade	Estrutura do slot
Distribuição	Sai embrulhada em key block, ou nunca sai	TR-31, KBK/KBK, <code>exportability</code>
Armazenamento	BRAM interna; backup como key block	Fronteira, hierarquia sob LMK
Uso	Só operações permitidas pelo modo de uso	Mediação, máquina de estados
Rotação	Nova chave gerada, antiga marcada	<code>use_count</code> , política
Destruição	Zeroize, com prova	<code>wipe()</code> , verificação por dump

A parte que quase sempre é subestimada é a **destruição**. Uma chave que “não é mais usada” mas continua num backup de fita, num slot que ninguém limpou, ou na memória de um módulo que foi para o lixo, continua sendo uma chave viva do ponto de vista do atacante.

Parte III — Como HSMs são atacados

Esta parte é o contrapeso da anterior. Cada mecanismo descrito até aqui existe porque alguém, em algum momento, quebrou um dispositivo que não o tinha.

Os ataques abaixo são **públicos e históricos**, documentados em literatura acadêmica e usados há décadas no ensino de segurança. Estão aqui pelo motivo de sempre: não se projeta defesa sem entender o ataque.

15. Ataques de API — o mais importante

A observação central, e a que mais custa a ser internalizada:

*Contra um HSM bem construído, você não ataca a criptografia. Você ataca a **API** — procura uma sequência de comandos, todos legítimos, que combinados revelem algo que nenhum comando isolado revelaria.*

O módulo funciona perfeitamente. Cada chamada é válida. Nenhum algoritmo é quebrado. E ainda assim a chave sai.

Este é o tipo de ataque que mais aparece contra HSMs reais, e o menos intuitivo para quem vem de criptografia teórica — porque a falha não está na matemática, está na **composição** de operações.

15.1 Confusão de tipo de chave

Nos formatos antigos da IBM CCA, o tipo de uma chave era codificado fazendo-se XOR de um *vetor de controle* com a chave mestra antes de cifrar a chave de trabalho. O tipo, portanto, não era autenticado — era apenas um valor combinado.

Mike Bond (2001) mostrou famílias de ataques que consistiam em manipular esses vetores para **converter uma chave de um tipo em outro**. Uma chave destinada a derivar PINs podia ser rerepresentada ao módulo como chave de dados; a partir daí, bastava pedir ao HSM que cifrasse com ela para obter o que deveria ser secreto.

A defesa é o TR-31, e agora a seção 8 faz sentido: o cabeçalho declara o tipo *e é coberto pelo MAC*. Alterar o tipo invalida o bloco inteiro. Toda a feiura do formato existe por causa desta classe de ataque.

15.2 Ataque da tabela de decimalização

O exemplo mais didático que existe, porque a matemática é trivial e o efeito é devastador.

O método de verificação de PIN IBM 3624 funciona assim: cifra-se o número da conta com a chave de derivação, obtêm-se dígitos hexadecimais, e uma **tabela de decimalização** os mapeia para decimais:

hex:	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
tabela:	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5

(padrão)

O problema: algumas APIs permitiam que **o chamador fornecesse a tabela**.

Bond e Zieliński (2003) observaram que, ao fornecer uma tabela degenerada — por exemplo, toda de zeros exceto uma posição — o atacante aprende, a cada consulta, se um determinado dígito aparece no PIN. Poucas consultas revelam o conjunto de dígitos; poucas mais revelam a ordem.

O resultado: recuperação de um PIN de quatro dígitos em cerca de **quinze chamadas de API**, contra as cinco mil de uma busca exaustiva. E cada chamada é perfeitamente legítima.

A lição de projeto: todo parâmetro controlado pelo chamador é uma entrada de ataque, inclusive os que parecem configuração inofensiva. É a razão do item mais importante do checklist deste projeto:

O que este comando pode fazer vazar se for chamado em laço com entradas escolhidas?

Note que a pergunta não é “este comando vaza a chave?”. É “**dez mil chamadas** deste comando vazam a chave?”.

15.3 Ataques a PIN blocks

Um PIN não trafega sozinho: ele vai dentro de um *PIN block*, que combina o PIN com o número da conta. O formato mais comum, ISO-o (ANSI X9.8), faz XOR entre o PIN preenchido e parte do número da conta.

HSMs de pagamento precisam **traduzir** PIN blocks — de uma chave para outra, de um formato para outro — porque a rede de pagamentos tem domínios de chave distintos entre adquirente, bandeira e emissor.

Clulow (2003) e outros mostraram que a combinação de tradução de formato com controle sobre o número da conta fornecido permite extrair informação sobre o PIN, um pedaço de cada vez. O módulo nunca revela o PIN; ele revela se uma operação teve sucesso, e isso basta.

A lição: funções de conversão são pontos de vazamento. Sempre que um módulo traduz entre representações, ele pode virar oráculo — e o mesmo padrão aparece fora de pagamentos, no *padding oracle*, em que um servidor que distingue “preenchimento inválido” de “MAC inválido” entrega decifração completa a quem perguntar o suficiente.

É por isso que, neste projeto, o arquivo de códigos de status carrega esta regra:

```
/* Um status nunca pode revelar mais do que o host tem direito de saber.  
"Chave errada" e "slot vazio" devem ser o mesmo código sempre que a  
distincao ajudar um atacante a enumerar o key store. */
```

15.4 O padrão comum

Repare no que os três têm em comum:

1. Todas as chamadas são **legítimas** — nenhuma validação foi burlada.
2. O vazamento vem da **composição**, não de um comando isolado.
3. O canal é estreito (um bit de sucesso/falha), e mesmo assim suficiente.
4. A defesa está no **projeto da API**, não em criptografia mais forte.

E é por isso que “todo opcode novo precisa de entrada na tabela, verificação de estado e teste, nessa ordem” é regra de projeto e não burocracia. A hora barata de perceber que um comando vaza é antes de escrevê-lo.

16. Canais laterais

Aqui o atacante não fala com a API. Ele **observa o dispositivo trabalhando** e infere o segredo a partir de algo que não era para ser saída.

16.1 Tempo

Kocher (1996) mostrou que o tempo de execução de exponenciação modular depende dos bits do expoente. Medindo tempos de resposta, recupera-se a chave privada.

A versão moderna mais comum não envolve matemática de chave pública: é **comparação de MAC que retorna cedo**.

```
/* ERRADO: vaza o comprimento do prefixo correto */  
if (memcmp(mac_calculado, mac_recebido, 16) != 0) return ERRO;
```

`memcmp` retorna assim que encontra diferença. Um atacante mede o tempo, descobre quantos bytes iniciais acertou, e reconstrói o MAC byte a byte — $2^{16} \cdot 256$ tentativas em vez de 2^{128} .

A defesa é **comparação em tempo constante**: acumular diferenças em vez de retornar cedo.

```
uint8_t diff = 0;  
for (int i = 0; i < 16; i++) diff |= a[i] ^ b[i];  
return diff == 0;
```

16.2 Cache

Uma variação que merece destaque porque explica uma decisão deste projeto. Implementações de AES por tabela têm tempo de acesso dependente do dado: se a entrada da tabela está em cache, é rápida; se não, lenta. E o índice da tabela depende da chave.

Bernstein (2005) e Osvik–Shamir–Tromer (2006) demonstraram recuperação de chave AES a partir apenas de temporização de cache, inclusive remotamente.

É por isso que neste projeto **as caches do processador estão desligadas**. Elas seriam inúteis (a memória interna já responde em um ciclo) e nocivas (introduzem tempo dependente do dado). Duas razões independentes, ambas suficientes.

16.3 Consumo de energia

Kocher, Jaffe e Jun (1999) introduziram a análise de potência:

SPA (Simple Power Analysis) — observar um único traço de consumo e ler a estrutura da operação. Numa exponenciação ingênua, quadrado e multiplicação têm assinaturas diferentes, e o traço *desenha* o expoente.

DPA (*Differential Power Analysis*) — coletar milhares de traços com entradas variadas e correlacionar estatisticamente com uma hipótese sobre alguns bits da chave. A hipótese certa produz correlação; as erradas, ruído. Funciona mesmo quando o sinal é muito menor que o ruído, porque a estatística acumula.

DPA é devastadora contra implementações desprotegidas e é a razão de existir uma indústria inteira de contramedidas: **maskamento** (dividir valores intermediários em partilhas aleatórias), **blinding** (randomizar a entrada de forma reversível), **embaralhamento** da ordem de operações, lógica de trilha dupla, e geradores de ruído.

16.4 Emissão eletromagnética

Mesmo princípio da análise de potência, mas captado por sonda próxima em vez de resistor em série. Vantagem para o atacante: é **local** — dá para posicionar a sonda sobre um bloco específico do chip e isolar o sinal que interessa, o que é especialmente eficaz contra circuitos grandes.

16.5 Onde este projeto está

Sem defesa nenhuma de canal lateral, e vale ser explícito: o AES que a Fase 2 vai instanciar no fabric não é mascarado, o firmware não faz comparação em tempo constante em todo lugar, e não há gerador de ruído.

O que existe é o começo certo: caches desligadas, e uma consciência de que o problema existe. A Fase 6 do plano ataca o próprio dispositivo, e é lá que esse assunto vira prático.

17. Injeção de falha

Se observar não basta, **estrague o cálculo e observe o erro**. É uma das famílias de ataque mais eficientes contra hardware criptográfico.

17.1 O ataque Bellcore

O exemplo mais elegante da área. Boneh, DeMillo e Lipton (1997) mostraram que **um único erro** numa assinatura RSA feita com o Teorema Chinês do Resto revela a fatoração da chave.

O RSA-CRT calcula duas metades, uma módulo p e outra módulo q , e as combina. Se uma das metades sair errada por causa de uma falha induzida, a assinatura resultante s satisfaz uma congruência e não a outra. Então:

$$\gcd(s^e - m \bmod N, N) = p$$

Uma assinatura defeituosa, um cálculo de máximo divisor comum, e a chave privada de 2048 bits está fatorada.

Isso é imensamente relevante para a Fase 7 deste projeto, que implementa RSA-2048 por Montgomery nos DSPs. **A contramedida padrão é verificar a assinatura antes de emití-la** — gastar uma exponenciação pública (barata, expoente pequeno) para garantir que a privada saiu certa.

17.2 Análise diferencial de falhas em cifras de bloco

Para AES, Piret e Quisquater (2003) mostraram que uma falha injetada nas últimas rodadas restringe drasticamente o espaço de chaves: poucos pares correto/defeituoso bastam para recuperar a chave de rodada, e dela a chave mestra.

17.3 Como se injeta a falha

Método	Custo	O que faz
Glitch de clock	muito baixo	Pulso curto demais faz a CPU pular ou corromper uma instrução
Glitch de tensão	baixo	Queda breve na alimentação, mesmo efeito
Pulso eletromagnético	médio	Sem contato, localizado
Laser	alto	Precisão de célula individual, exige decapsulação
Temperatura	baixo	Fora de faixa, memórias e lógica ficam instáveis

O glitch de clock é o mais didático e o mais barato. Se o pulso cair no instante certo, a CPU pula uma instrução — e se a instrução pulada for o `if` que compara o PIN, ou o que verifica a exportabilidade da chave, você contornou a verificação sem tocar em criptografia alguma.

17.4 Contramedidas

- **Dupla execução e comparação** — calcular duas vezes e confrontar. Dobra o custo e derrota falhas isoladas.
- **Verificação do resultado** — no RSA, checar a assinatura antes de emitir.
- **Sensores** — detectores de frequência fora de faixa, de tensão, de temperatura, de luz. Ao disparar: zeroize.
- **Redundância na codificação** — representar estados críticos com valores distantes em distância de Hamming, para que um bit trocado não converta “recusado” em “aceito”.

O último ponto é subestimado. Se `AUTORIZADO = 1` e `NEGADO = 0`, um único bit trocado inverte a decisão. Se `AUTORIZADO = 0x5A` e `NEGADO = 0xA5`, é preciso trocar muitos bits coordenadamente.

17.5 Onde este projeto está

O gerador de clock deste projeto tem a versão mínima e honesta de um sensor:

```
wire rst_src_n = locked_w & rst_n_sync[1];
```

Perda de lock do MMCM **reafirma o reset** em vez de ser ignorada. Não sabemos *por que* o lock caiu, mas sabemos que não dá para confiar no que vier depois. Um HSM real iria além: zeroizaria, iria para `TAMPERED`, e registraria o evento.

A Fase 6 do plano faz exatamente este ataque contra o próprio dispositivo — glitch de clock pela porta de reconfiguração dinâmica do MMCM — e depois implementa as contramedidas. É o roteiro clássico de aprender segurança de hardware: construir, quebrar, reforçar.

18. Ataques físicos e antitamper

O degrau mais alto da escada: o atacante tem o aparelho na bancada.

18.1 A escada da resistência

Nível	Nome	O que significa
1	Tamper-evident	A violação deixa marca visível: lacre, resina que estilhaça, tinta que mancha
2	Tamper-resistant	A violação é difícil: resina dura, blindagem, parafusos especiais
3	Tamper-responsive	O dispositivo detecta e reage : malha ativa que zeroiza ao ser rompida
4	Environmental failure protection	Detecta condições anormais — temperatura, tensão — e zeroiza antes de falhar

O salto conceitual está entre 2 e 3. Resistência apenas ganha tempo; **resposta** destrói o alvo. Um HSM de alto nível tem uma malha de trilhas finíssimas envolvendo a eletrônica, monitorada continuamente. Furar, esmerilhar ou dissolver a resina rompe a malha, e a chave desaparece antes que se chegue nela.

18.2 Por que a bateria importa

A malha precisa estar viva mesmo com o equipamento desligado — senão o ataque é trivial: corte a energia, abra com calma. Por isso HSMs de alto nível têm bateria interna que mantém detectores e memória de chaves.

E daí decorre um comportamento que parece defeito e é projeto: **um HSM cuja bateria descarregou acorda zeroizado e inútil**. É a falha segura correta.

Este projeto reproduz isso por acidente de hardware, e o acidente é instrutivo: no core board QMTECH usado, o pino VCCBATT está ligado ao rail de 1,8 V, sem bateria. A chave de bitstream em BBRAM se perde a cada desligamento e precisa ser recarregada por JTAG.

Aqui isso é desejável — nada persiste no hardware, tudo é recuperável — e é exatamente o comportamento de um HSM comercial com bateria descarregada.

18.3 Ataques invasivos

- **Decapsulação** — remover o encapsulamento com ácido ou plasma, expondo o die.
- **Microsondagem** — colocar sondas em trilhas internas e ler barramentos diretamente.

- **FIB** (*Focused Ion Beam*) — cortar e refazer ligações no próprio die; permite, por exemplo, desconectar um sensor.
- **Imageamento** — ler ROM ou fusíveis opticamente, ou por microscopia eletrônica.
- **Cold boot** — resfriar a memória para retardar a perda de conteúdo e lê-la em outro equipamento. Relevante para DRAM, e mais um motivo para chaves não morarem em memória externa.

18.4 O que este projeto não tem

Absolutamente nada disso. Não há resina, malha, sensor, bateria ou blindagem. O FPGA está numa placa de desenvolvimento com conectores de expansão expostos.

Vale registrar sem rodeios: **o modelo de ameaça deste projeto vai até o adversário que fala com a API**. Contra qualquer coisa acima disso, ele não oferece defesa — e não pretende.

O valor está em entender por que os degraus existem. Um leitor que termine esta parte sabendo *o que* uma malha ativa faz, *por que* ela precisa de bateria e *por que* um HSM sem bateria acorda inútil aprendeu algo que nenhum diagrama de blocos ensina.

Parte IV — Certificação

Um HSM comercial não é vendido pelo que faz, e sim pelo que **um terceiro verificou** que ele faz. Esta parte explica o que os selos significam — e, igualmente importante, o que não significam.

19. FIPS 140-3

A norma dos Estados Unidos para módulos criptográficos, hoje alinhada à ISO/IEC 19790. Substituiu a FIPS 140-2, e a validação é feita pelo CMVP (programa conjunto NIST/CCCS) por meio de laboratórios credenciados.

A norma define **quatro níveis**, e a diferença entre eles é quase toda física.

Nível 1

Requisitos mínimos. Algoritmos aprovados, implementados corretamente. **Sem exigência de segurança física** — uma biblioteca de software rodando num PC comum pode ser validada em nível 1.

Utilidade real: garante que o AES é AES de verdade e que os testes de resposta conhecida existem. Não diz nada sobre proteger a chave de quem tem acesso à máquina.

Nível 2

Acrescenta **evidência de violação**: lacres, revestimentos ou invólucros que mostrem que alguém abriu. E autenticação **por papel** — o módulo distingue “operador” de “administrador”.

A palavra-chave é *evidência*: o nível 2 não impede a violação, garante que ela deixe marca.

Nível 3

Aqui a coisa fica séria. Acrescenta:

- **Resistência e resposta à violação** — o módulo detecta a intrusão e **zeroiza** os parâmetros críticos de segurança;
- **autenticação por identidade** — não basta o papel, é preciso saber quem;
- **separação física ou lógica** das interfaces por onde entram e saem parâmetros críticos.

A maior parte dos HSMs comerciais de propósito geral é validada em nível 3.

Nível 4

O topo. Um **envelope completo de proteção**: qualquer tentativa de acesso físico, de qualquer direção, é detectada e responde com zeroização. E acrescenta **proteção contra falhas ambientais** — o módulo detecta tensão ou temperatura fora da faixa e zeroiza antes que a condição possa ser explorada.

O nível 4 é caro, e existem poucos produtos. É a resposta direta aos ataques descritos na seção 17: se você pode congelar o chip ou baixar a tensão para induzir falhas, o módulo precisa perceber e se destruir primeiro.

O vocabulário que a norma impõe

Vale conhecer, porque aparece em toda documentação da área:

- **CSP** (*Critical Security Parameter*) — chave em claro, dado de autenticação, qualquer coisa cuja divulgação comprometa a segurança.
- **Estado de erro** — condição em que o módulo entra ao falhar um autoteste e na qual **não realiza operações criptográficas**.
- **Modo aprovado** — o conjunto de operações e algoritmos cobertos pela validação. Um módulo pode ter funções fora do modo aprovado; usá-las tira você da conformidade.
- **Autotestes** — os KAT da seção 11, exigidos na inicialização e sob condições específicas.

20. PCI: as regras de pagamentos

Enquanto FIPS avalia o **módulo**, o mundo de pagamentos regula também **como ele é operado**.

PCI PTS HSM define requisitos de segurança para o equipamento usado em processamento de PIN e chaves de pagamento.

PCI PIN Security Requirements cobre a operação: como chaves são geradas, transportadas, carregadas, trocadas e destruídas. É daqui que vêm, como exigência contratual e não como boa prática:

- **dual control** — nenhuma operação sensível de chave por uma pessoa só;
- **split knowledge** — nenhuma pessoa conhece uma chave inteira;
- **cerimônia documentada**, com testemunhas e registro;
- **key blocks** para transporte de chave — e há prazos de migração obrigatória para formatos como o TR-31.

É por isso que este projeto modela LMK, componentes e dual control com botões físicos: no mundo de pagamentos esses mecanismos são **requisito auditado**, não escolha de arquitetura.

21. Common Criteria

Norma internacional (ISO/IEC 15408) com abordagem diferente: em vez de níveis fixos, define **perfis de proteção** — descrições do que um tipo de produto deve fazer — e avalia produtos contra eles, com um nível de garantia (**EAL 1 a 7**) que indica o rigor da avaliação, não a força da segurança.

Confusão comum, e vale desfazer: **EAL alto não significa “mais seguro”**. Significa “avaliado com mais rigor e mais documentação formal”. Um produto simples avaliado em EAL 6 pode proteger menos que um produto rico em EAL 4 — o que o EAL mede é a confiança na avaliação, não a altura do muro.

Na Europa, perfis como o EN 419221-5 cobrem módulos criptográficos para serviços de confiança, e são o caminho para dispositivos de assinatura qualificada sob o eIDAS.

22. O que “certificado” significa — e o que não significa

Esta seção é a mais importante da Parte IV.

Significa

- Um laboratório independente verificou as afirmações do fabricante.
- Os algoritmos foram testados contra vetores oficiais.
- Existe documentação de fronteira, de estados, de papéis e de zeroização.
- Há um processo de reavaliação quando o produto muda.

Não significa

Não significa “inviolável”. Certificação estabelece um piso verificado, não um teto. Módulos certificados já foram quebrados — e certificados são revogados ou emendados quando isso acontece.

Não significa que a sua configuração está segura. A validação cobre o módulo operando em **modo aprovado**, numa configuração específica descrita na política de segurança. Usá-lo fora dali é usar um produto não validado com um adesivo de validado.

Não significa que o sistema em volta está seguro. O HSM protege a chave. Se a aplicação assina qualquer coisa que peçam, o atacante não precisa da chave — ele pede a assinatura. Essa é, na prática, a falha mais comum em implantações reais: um HSM impecável atrás de uma API de aplicação que autoriza demais.

Não significa nada sobre canal lateral, salvo se explicitado. Resistência a DPA é avaliada separadamente e nem sempre está incluída.

A moral para este projeto

Não há nenhuma pretensão de certificação aqui, e nem poderia haver. Mas conhecer a estrutura das normas explica **por que** as fases do plano estão na ordem em que estão:

- os autotestes da Fase 2 são o requisito de autoteste do FIPS;
- a máquina de estados da Fase 3 é a separação de papéis;
- o zeroize com prova é o requisito de zeroização;
- o log de auditoria é a evidência;
- os sensores da Fase 6 são o nível 3/4 de resposta física.

Implementar cada mecanismo e depois ler o parágrafo da norma que o exige é uma forma muito eficiente de entender a norma — e de perceber que ela não é burocracia arbitrária, mas cicatriz acumulada.

Parte V — O projeto hsm-a7

23. Objetivo, plataforma e restrições

O objetivo

Construir, do zero, um módulo criptográfico com hierarquia de chaves, cerimônia de carga, key blocks e API de host — reproduzindo a arquitetura de um HSM real em escala reduzida, para **entendê-la de dentro para fora**.

A escolha de fazer em FPGA, e não em software, é deliberada: obriga a enfrentar a fronteira física, a distinguir memória interna de externa, e a lidar com clock e reset como questões de segurança e não de infraestrutura.

A plataforma

Item	Valor
FPGA	Xilinx Artix-7 XC7A35T , encapsulamento FTG256, speed grade -1
Placa	QMTECH core board + daughterboard
Recursos	20.800 LUTs, 41.600 flip-flops, 50 BRAMs, 90 DSP48E1
Clock	50 MHz no pino N11 → 100 MHz por MMCM
Processador	NEORV32 (RISC-V RV32IMC), submódulo fixado em v1.13.3
Canal	UART 115200 8N1 pelo CP2102 do core board
Flash	MT25QL128, 16 MB, a mesma que guarda o bitstream

O orçamento de fabric é apertado de propósito: AES, SHA, key store e processador precisam caber em 20.800 LUTs, e isso força decisões explícitas em vez de acumulação.

As restrições invioláveis

Estas não são preferências. Estão no topo do plano e do arquivo de instruções do projeto.

- 1. Nada de eFUSE.** Não gerar, sugerir ou executar `program_efuse`, `CFG_AES_ONLY`, `W_DIS` / `R_DIS` ou qualquer variante. São memórias de programação única: o erro é um *brick* permanente. Chave de bitstream apenas em BBRAM, que é recuperável.
- 2. Chaves em claro só em BRAM.** Nunca DDR3, nunca flash sem embrulho, nunca um registrador exposto no toplevel.
- 3. Nada atravessa a fronteira em claro.** Saem apenas key blocks embrulhados, KCVs, criptogramas, *handles* e log.

4. Simulação antes de hardware. Nenhum módulo vai para a placa sem testbench passando. Depurar criptografia por UART, sem visibilidade interna, é a forma mais lenta possível de descobrir que faltou um byte no preenchimento.

5. Não enfraquecer testes para fazer algo passar. Se um KAT falha, o bug está no código, não no vetor.

A regra do eFUSE merece um comentário, porque ela ilustra um princípio geral: **preferir sempre o mecanismo recuperável ao irreversível, num projeto de estudo.** O JTAG também nunca é desabilitado — é a única via de recuperação, e um projeto de aprendizado precisa poder voltar atrás. Um produto real faria o oposto em ambos os casos, e entender por que os dois caminhos são corretos em contextos diferentes é parte do conteúdo.

24. Fase 1, peça por peça

Esta seção pega cada arquivo escrito e responde: **o que essa peça corresponde num HSM real?**

Cada uma termina com *o que ainda falta*, porque a distância entre o brinquedo e o produto é o conteúdo das fases seguintes.

Mapa

O que construímos	O que é num HSM real
<code>clk_rst_gen.v</code>	Monitor de integridade do clock (sensor antitamper)
<code>debounce.v</code>	Integridade da autorização física (dual control)
<code>neorv32_wrapper.vhd</code>	Endurecimento da plataforma: o que a máquina não faz
<code>hsm_top.v</code>	Definição física da fronteira criptográfica
<code>cmd.c</code>	A API — a superfície de ataque principal
CRC32	Integridade de transporte (num HSM real, viraria MAC)
<code>state.c</code>	A chave de switch de três posições
<code>wipe.c</code>	Requisito de zeroização do FIPS 140-3
LEDs D1–D5	Indicadores de estado e de violação
<code>BOOT_MODE_SELECT=2</code>	Autenticidade de firmware / secure boot
IMEM como ROM	Integridade do código em execução
<code>tb_soc_silent</code>	Higiene de canal lateral
<code>hsmtool.py</code>	O cliente da API — futuro PKCS#11
Teste de 10.000 pings	Requisito de confiabilidade operacional

24.1 `clk_rst_gen.v` — o clock é um sensor

MMCM que transforma 50 MHz em 100 MHz (VCO em 1000 MHz, meio da faixa do speed grade -1), mais geração de reset: afirma assíncrono, libera síncrono, segura 64 ciclos após o lock.

Num HSM real: monitoramento de clock é sensor antitamper. Ver seção 17.3 — glitch de clock é o ataque de injeção de falha mais barato que existe, e faz a CPU pular exatamente a instrução que verifica alguma coisa.

O ponto de projeto está numa linha:

```
wire rst_src_n = locked_w & rst_n_sync[1];
```

Perda de LOCKED **reafirma o reset**. Não sabemos por que o lock caiu, mas sabemos que não dá para confiar no que vier depois.

O gerador é também a única exceção à convenção de reset síncrono do projeto, e por necessidade: com o MMCM em reset, o clock de saída não está correndo e não existe borda para registrar nada. Usa o padrão clássico *afirma assíncrono / libera síncrono*, justamente para que todo o resto do projeto possa usar reset puramente síncrono.

Falta: zeroizar e ir para TAMPERED, registrar o evento, e monitorar a frequência ativamente em vez de só reagir à perda de lock. Fase 6.

24.2 `debounce.v` — quando um ressalto vira autorização

Exige 10 ms de estabilidade contínua antes de aceitar mudança; qualquer transição no meio zera a contagem.

Num HSM real: integridade da autorização física (seção 7). O mecanismo precisa refletir intenção humana, não estado elétrico de contato.

Os dois botões escolhidos são SW2 e SW5 — os extremos da fileira — para dificultar pressionar ambos com uma mão só.

Falta: autenticação de custodiante (smartcard + PIN) e registro de *quem* autorizou, não só de que houve autorização.

24.3 `neorv32_wrapper.vhd` — segurança é o que se desliga

Fixa os ~130 generics do processador. É o único lugar onde a configuração do SoC está escrita — e num relatório de avaliação, é exatamente essa lista que o laboratório examina.

Desligado	Por quê
OCD_EN	Debugger dá leitura e escrita irrestritas de memória por JTAG. Com a LMK em BRAM, é extração de chave sem passar pela API. O generic mais importante do arquivo.
ICACHE / DCACHE	Inúteis (BRAM já responde em 1 ciclo) e nocivas: tempo dependente do dado é canal lateral (seção 16.2).
XBUS / SMC	Sem barramento externo, não há caminho físico até a DDR3. A regra vira estrutura.
IO_TRACER	Buffer de rastreamento de execução é observabilidade indesejada.
Zkne / Zknd / Zknh	AES e SHA vão para o fabric via CFS: construí-los é o objetivo do projeto.

Sobre desligar o debugger, note a assimetria com a regra de nunca desabilitar JTAG no FPGA: lá é o caminho de recuperação do **bitstream**, e precisa existir num projeto de estudo. Aqui é o caminho de leitura da **memória do processador**, e não precisa. Duas coisas diferentes com o mesmo nome.

Ligados: RV32IMC (RISC_V_ISA_C + _M), contadores base, barrel shifter (código criptográfico é pesado em deslocamento), IMEM 16 KB, DMEM 8 KB, UART0 com FIFO de 64, temporizador, GPIO de 8 bits.

Falta: PMP (proteção de memória física) para separar privilégio dentro do firmware, de modo que um bug no parser não alcance a região da LMK. Fase 3.

24.4 hsm_top.v — onde a fronteira fica física

Único arquivo do projeto que conhece nome de pino. Corresponde à **definição da fronteira criptográfica** — o desenho que num documento FIPS marca dentro e fora, com a lista de todas as portas.

Aqui a lista é curta e verificável:

Porta	Direção	O que atravessa
uart_rxd_i / uart_txd_o	ambas	comandos e respostas
btn_a_i / btn_b_i	entra	autorização física
led_o , seg_o , seg_an_o	sai	estado

E só. Não existe porta de dados além da UART, e o que passa por ela é definido pela tabela de comandos.

O toplevel também absorve, num ponto só, as duas inversões da placa — LEDs e botões são ativos em nível baixo. Parece cosmético e não é: **um LED de TAMPERED invertido é defeito de segurança**, porque o dispositivo mostraria “tudo bem” exatamente quando não está.

24.5 `cmd.c` — o parser é a superfície de ataque

Máquina de estados de recepção, verificação de CRC, tabela de comandos com verificação de estado, montagem da resposta.

Num HSM real: esta é a API, e é onde mora a maior parte dos ataques reais (seção 15).

Por isso o checklist obrigatório para todo opcode novo:

- Em quais estados é permitido?
- Exige dual control?
- **O que pode vazar se for chamado em laço com entradas escolhidas?**
- Respeita a exportabilidade do slot?
- Entra no log **antes** da execução?

O parser é deliberadamente burro: máquina de estados explícita, todos os limites verificados, sem alocação dinâmica, sem recursão. É o único código que processa entrada arbitrária antes de qualquer verificação — não é lugar para esperteza.

O timeout entre bytes é o que satisfaz o critério “frame malformado nunca trava a máquina de estados”. Sem ele, um host que morre no meio de um frame deixa o parser esperando para sempre e o dispositivo mudo: negação de serviço com um único byte enviado. É rearmado a cada byte, e não por frame, porque um key block TR-31 leva cerca de 45 ms a 115200 baud e um timeout de frame teria de ser afrouxado até deixar de proteger.

Disponibilidade é requisito num HSM: um módulo que para de responder derruba a autorização de transações.

Falta: o log antes da execução (o ponto está marcado no código), e tempo de resposta constante — hoje um comando recusado responde mais rápido que um aceite, e isso é informação.

24.6 O CRC32, e por que num HSM real seria um MAC

Detecta corrupção de enquadramento. Polinômio IEEE 802.3 refletido, idêntico ao `zlib.crc32`.

Distinção que vale internalizar: CRC é detecção de erro, **não** autenticação. Quem altera o payload recalcula o CRC trivialmente.

Num HSM de pagamentos, o canal com o host costuma ser autenticado de verdade — MAC sobre a mensagem com chave de sessão, ou TLS mútuo.

Aqui o CRC basta, e o motivo é conceitualmente importante: **a fronteira de confiança está no dispositivo, não no canal**. Assumimos que o host pode ser hostil, e um host hostil não precisa falsificar CRC — ele manda comandos válidos. A defesa não é autenticar o canal; é a máquina de estados, o dual control físico e os atributos de chave.

É por isso que dual control é feito com botões na própria placa e não com um comando “eu autorizo” vindo pelo cabo.

Duas decisões de formato que o plano deixava em aberto e precisavam ser fixadas antes de o host existir:

- **o CRC cobre LEN + corpo** — incluir o `LEN` importa, porque é o campo que um atacante usaria para induzir leitura fora do buffer;
- **polinômio idêntico ao `zlib.crc32`** — mantém o host trivial e elimina uma fonte clássica de divergência.

O CRC existe em três implementações independentes: C no firmware, Verilog no testbench, Python no host. Divergência apareceria como erro de CRC intermitente — dos sintomas mais enganosos que existem, porque parece problema de cabo.

24.7 `state.c` — a chave de switch

Hoje só o enum e a consulta: o dispositivo nasce e permanece em `UNINITIALIZED`.

Existe agora, com um estado só, porque **a tabela de comandos já carrega a máscara de estados permitidos**. Retroajustar verificação de estado em cada handler depois é como se esquece um.

```
#define ST_NORMAL (ST_UNINIT | ST_AUTH | ST_OPER)
```

`TAMPERED` fica fora de propósito: dispositivo comprometido responde o mínimo possível.

24.8 `wipe.c` — zeroização antes de haver chave

Ponteiro `volatile` mais barreira de memória, pelo motivo explicado na seção 12.

Está no lugar mesmo a Fase 1 não movimentando chave nenhuma, e os buffers de frame são limpos após cada comando. Motivo: são **esses mesmos buffers** que vão carregar componente de LMK e key block na Fase 3.

Falta: o `ZEROIZE` de verdade, com sobrescrita por padrão e depois zeros, e o teste que faz dump da região e **prova** que apagou.

24.9 Os LEDs, e por que D1 é em hardware

D1 pisca a 1 Hz por lógica no fabric; D2 a D5 são GPIO controlado pelo firmware.

A decisão interessante: **D1 é independente da CPU**. Se o firmware travar, D1 continua piscando. Isso distingue duas falhas que de fora parecem iguais:

Sintoma	Diagnóstico
D1 parado	clock morto — MMCM sem lock, alimentação, bitstream
D1 piscando, D2 apagado	clock bom, firmware pendurado
D1 piscando, D2 aceso	plataforma viva; o problema é protocolo ou host

Num dispositivo sem console, essa é a diferença entre depurar e adivinhar. É o mesmo princípio do watchdog independente: o indicador de vida não pode depender da coisa cuja vida ele indica.

Bônus de verificação: D1 a 1 Hz cronometrado a olho **confirma a divisão do MMCM**, porque o contador é `CLK_HZ/2` e só bate 1 Hz se o clock for mesmo 100 MHz.

24.10 `BOOT_MODE_SELECT = 2` — autenticidade de firmware

O SoC sobe a partir de uma imagem embutida na IMEM, não do bootloader do NEORV32.

Num HSM real: isto é *secure boot*, um dos requisitos mais duros de certificação.

O bootloader do NEORV32 aceita upload de firmware pela UART. Com ele ligado, qualquer um com acesso ao serial substitui o firmware — e firmware substituído contorna a máquina de estados, o dual control, a exportabilidade e a fronteira inteira. **Todas as defesas discutidas neste documento passam a valer zero**, porque quem escreve o código escreve as regras.

Num HSM comercial isso se resolve com boot ROM imutável que verifica assinatura da imagem antes de executá-la, com a chave pública em memória de programação única.

A versão deste projeto é mais simples e mais rude: **não há caminho de atualização**. O firmware está no bitstream; trocá-lo exige regerar o bitstream e gravar por JTAG, o que é acesso físico.

Efeito colateral que veio de graça: com esse modo, o NEORV32 liga `MEM_INIT` e a IMEM vira memória **somente de leitura**. A memória de código não é gravável em tempo de execução — não há injeção de código nem código automodificável, independentemente de qualquer bug no parser.

Isso é integridade de código em execução. Num sistema com MMU seria o equivalente a marcar as páginas de texto como não-graváveis; aqui é mais forte, porque não é uma permissão que alguém pode mudar — é a ausência de circuito de escrita.

Também sumiu a boot ROM: a BRAM caiu de 8 tiles para 3,5.

24.11 `tb_soc_silent` — não falar sem ser perguntado

Verifica que o dispositivo não transmite nada pela UART até receber um comando — **e** que está vivo.

Num HSM real: higiene de canal lateral e de vazamento de identidade. Um banner de boot entrega versão e identidade a quem só escuta o barramento, o que é reconhecimento gratuito. E um `printf` de depuração esquecido no firmware é um canal lateral permanente, historicamente uma das formas mais comuns de vazamento em dispositivos embarcados.

O detalhe que torna o teste honesto: silêncio sozinho não prova nada, porque firmware travado também fica quieto. Por isso o teste exige silêncio **e** vida — `main()` precisa ter chegado ao laço e aceso o LED de atividade. Um teste que passa quando o dispositivo está morto é pior que teste nenhum.

24.12 `hsmtool.py` — o cliente da API

Codec do protocolo, transporte serial, CLI. Em produção isso seria PKCS#11 ou um command set ASCII estilo payShield — é o que a Fase 5 constrói.

O comentário no topo do arquivo é a regra que o mantém honesto:

```
# FRONTEIRA: tudo aqui esta FORA. Este programa nunca ve chave em claro --
# so key blocks wrapped, KCVs, criptogramas, handles e log. Se algum comando
# futuro fizer material de chave aparecer neste arquivo, o comando esta
# errado, nao o script.
```

Aqui se entende por que a API de um HSM é do jeito que é: **é você que precisa impedi-la de vazar**. Quando se escreve os dois lados, a tentação de adicionar um “só para depurar, me devolva a chave” fica concreta — e recusá-la ensina mais do que ler que não se deve.

24.13 O teste de 10.000 pings

Num HSM real: disponibilidade e determinismo são requisitos operacionais. Um módulo autorizando transações tem acordo de nível de serviço; um que perde uma mensagem em dez mil produz falhas intermitentes que ninguém reproduz.

E há um aspecto de segurança: **erro intermitente esconde ataque**. Se o canal normalmente perde alguns frames, ninguém investiga o dia em que alguém está injetando tráfego. Uma taxa de erro de exatamente zero é o que torna qualquer erro digno de investigação.

25. Resultados medidos

Recursos e temporização

Recurso	Só clock/reset	Com o SoC	Com firmware	Disponível
Slice LUTs	24	2247	2240 (10,8%)	20800
Flip-flops	42	1633	1633 (3,9%)	41600
Block RAM	0	8	3,5 (7%)	50
DSP48E1	0	0	0	90
MMCM	1	1	1	5

Temporização final: **WNS +0,637 ns** em período de 10 ns, sem endpoints falhando em 4529. Fmax $\approx$ 107 MHz para os 100 MHz exigidos.

A margem já caiu duas vezes — de +5,95 ns com só o gerador de clock, para +0,898 com o SoC, para +0,637 com o firmware. O caminho crítico está dentro da CPU: banco de registradores em BRAM, onze níveis de lógica com sete cadeias de carry, de volta ao banco. Os cores de criptografia da Fase 2 são datapath separado e não alongam esse caminho, mas adicionam congestionamento — e roteamento já é 45% do atraso.

***Nota escrita depois, e vale mais que a previsão.** O parágrafo acima acertou o mecanismo e errou a conclusão. Os cores realmente não alongaram o caminho da CPU — ele continua passando. Mas trouxeram um caminho **próprio**, muito pior: -2,388 ns dentro do SHA-256, contra os +0,637 ns que se estava vigiando. A margem que importava não era a que estava sendo medida.*

A lição é sobre a forma do erro, não sobre o número: vigiar a métrica que se conhece é conforto, e o que quebra costuma vir de onde ainda não há métrica. A história completa está na seção 27.

Firmware: **1652 bytes de código, 1572 de dados não inicializados** — 10% da IMEM e 19% da DMEM. A maior parte da bss são os três buffers de frame.

Critério de aceitação, medido na placa

```
iteracoes : 10000 em 44,3 s (226/s)
erros      : 0
latencia   : min 3,97   mediana 4,36   p99 4,85   max 6,50 ms
```

Zero erros de CRC em dez mil iterações, e o pior caso 7,7 vezes abaixo do limite de 50 ms. A distribuição é apertada, sem cauda longa — cauda longa indicaria contenção ou retentativa escondida.

A latência é dominada pelo USB, não pelo dispositivo: dezesseis bytes a 115200 baud são 1,4 ms de tempo de linha, e o resto é o temporizador de latência do CP2102. Vale lembrar disso antes de “otimizar” o firmware por engano.

26. Três lições que a Fase 1 ensinou fazendo

Não estavam no plano. Vieram de erros reais.

Verificação encontra o que revisão não encontra

O testbench do toplevel reprovou a primeira versão com `led=1111x`. Registradores sem valor inicial mais reset síncrono deixam uma janela indefinida entre a configuração do FPGA e o primeiro clock.

Em hardware a ferramenta teria inicializado implicitamente e nada apareceria. Mas a saída em questão é um **indicador visível de estado** — e na Fase 3 uma delas é o LED de `TAMPERED`. Depender de comportamento implícito da ferramenta para o estado de um indicador de segurança é exatamente o tipo de coisa que um avaliador pergunta.

Cuidado ao “corrigir” um teste que falha

O testbench de boot reprovou esperando `\n` e recebendo `\r\n`. A tentação é ajustar a expectativa e seguir.

A conduta correta foi ir verificar na fonte do NEORV32 que o driver realmente expande LF em CRLF **antes** de mexer no teste. A diferença entre “corrigi uma expectativa errada” e “ajuste o teste ao resultado observado” é invisível no diff e enorme na prática — é a mesma disciplina da regra inviolável número 5.

Hardware expõe o que simulação estruturalmente não pode

A ferramenta de host escolhia a primeira porta serial encontrada. Na bancada isso é o **adaptador JTAG**, porque o gravador usado é um FT232H que também cria porta serial e enumera antes da placa.

O sintoma seria timeout sem explicação, apontando para o firmware quando o problema é o cabo. Nenhum teste sem hardware pegaria: o pseudo-terminal usado no teste de transporte não tem identificador USB, e o selftest do codec nem abre porta.

A lição não é “simulação é insuficiente” — a Fase 1 pegou dois defeitos reais em simulação e só um escapou. É saber **qual classe** de defeito cada método alcança, e não confundir “os testes passam” com “está correto”.

Parte VI — O caminho adiante

27. Fase 2 — engines e self-test (em andamento)

Objetivo: primitivas corretas e verificáveis. Correção antes de desempenho.

Bloco	Fonte	Estado
AES-256	secworks/aes	no CFS, verificado
SHA-256	secworks/sha256	no CFS, verificado
DNA_PORT	primitiva Xilinx	no CFS, verificado
TRNG	neoTRNG (do NEORV32)	a fazer
CTR_DRBG	firmware	a fazer

O que já está feito

Os vetores KAT foram baixados das fontes oficiais — NIST CAVP para AES e SHA, RFC 4231 para HMAC, SP 800-90A para o DRBG — com URL e hash SHA-256 de cada arquivo registrados num manifesto. Nada foi escrito de memória.

Isso não é formalidade. A regra inviolável do projeto diz que, se um KAT falha, o bug está no código e não no vetor — e essa regra só tem força se a procedência do vetor for verificável por um terceiro. Um script rebaixa, confere hash e reextrai; sobre um repositório limpo, o resultado é byte a byte idêntico.

Os dois cores passam:

AES-256	405 vetores × 4 (ECB/CBC, cifra/decifra) = 1620
SHA-256	65 mensagens, 74 blocos

O AESAVS traz quatro tipos de vetor e todos foram usados, porque cada um pega uma classe diferente de defeito: `VarKey` varre cada bit da chave isoladamente e acha indexação errada na expansão; `VarTxt` varre cada bit do bloco e acha ordem de byte e permutação trocadas. Um core que passa apenas no `GFSbox` pode estar inteiro errado.

Uma divisão de responsabilidade ficou registrada e vai importar no firmware: o core de AES faz ECB, e o encadeamento de CBC é de quem chama; o core de SHA recebe blocos de 512 bits já preenchidos, e o *padding* também é de quem chama. Testar o core com essas coisas embutidas misturaria falhas de naturezas diferentes.

O teste encontrou um defeito no próprio teste. O testbench de SHA reprovou com um sintoma característico: digests corretos, porém deslocados de um — a mensagem 1 devolve o resultado da 0. Esperar apenas o sinal de “pronto” logo após o comando termina de imediato, porque o núcleo ainda não o baixou, e lê-se o resultado anterior. O testbench de AES passava com o mesmo handshake fraco, por sorte de temporização; recebeu a mesma correção. Teste que passa por sorte é dívida, não aprovação.

O coprocessador: o CFS

Os cores entraram pelo **CFS** (*Custom Functions Subsystem*) do NEORV32 — um slot de periférico projetado para receber coprocessadores. O arquivo do upstream é um template feito para ser jogado fora; a substituição é feita trocando qual arquivo o build compila, sem patch e sem tocar no submódulo.

É também por onde o `GET_DNA` finalmente se resolveu, fechando a última sobra da Fase 1.

O CFS, e não um barramento externo, por um motivo de arquitetura: o coprocessador precisa estar **dentro da fronteira criptográfica** (seção 5), não pendurado num barramento que sai do die. Coerente com `XBUS_EN => false`, que é a mesma decisão vista do outro lado.

Três escolhas do mapa de registradores merecem nota, porque cada uma é um conceito da Parte II virando linha de RTL:

- **O único fio do CFS que chega ao toplevel está amarrado em zero.** Não existe caminho físico do bloco que guarda a chave até um pino. A fronteira deixa de ser promessa de firmware e vira topologia.
- **Chave e blocos de entrada são escrita-somente;** ler devolve zero. Não protege contra a CPU, que acabou de escrever a chave — evita criar um caminho de leitura que um bug use por acidente.
- **Não há AES-128.** O tamanho de chave é fixo no hardware. Um modo mais fraco alcançável por escrita em registrador seria um *downgrade* de graça, e a seção 15 mostra o que APIs fazem com modos mais fracos disponíveis.

E há um `WIPE` que não é decorativo: ele sobrescreve a **chave expandida** dentro do core, não só o registrador visível. O teste correspondente é o que separa zeroização de teatro de zeroização — depois do wipe, cifrar sem carregar chave nenhuma tem de produzir o resultado sob a chave zero.

*Um testbench a mais, para uma pergunta diferente. Os KAT de core provam que o AES está certo. Não provam que o caminho entre a CPU e o AES está certo — um core correto atrás de um mapa de registradores com a ordem das palavras invertida produz resultados perfeitamente errados. Por isso os mesmos vetores do NIST são replicados uma segunda vez, agora **através do barramento**. Foi assim que a Fase 1 já tinha aprendido a separar “o módulo funciona” de “o sistema funciona”.*

O preço em silício, e o que ele ensinou

Colocar criptografia no fabric custou caro e o custo apareceu onde não se esperava.

	Fase 1	Com o CFS
Slice LUTs	2240 (10,8%)	7035 (33,8%)
Flip-flops	1633 (3,9%)	6235 (15,0%)
Block RAM	3,5	3,5
DSP48E1	0	0

Área não foi problema — sobra um terço do chip. **Temporização foi.** O projeto vinha com +0,637 ns de folga e, com os cores dentro, foi para **-2,388 ns**: 58 caminhos falhando, o dispositivo não podia ser gravado.

O diagnóstico veio antes de qualquer conserto, e foi ele que economizou o trabalho: dos 58 caminhos, **49 estavam no SHA-256 e 9 no AES**, e o pior do AES era -0,087 ns — ruído. Um único bloco respondia por tudo. E dentro dele, um único caminho: **oito somas de 32 bits encadeadas num ciclo só**, porque a expansão da mensagem e a rodada de compressão dividem o mesmo ciclo.

A correção foi registrar as duas entradas combinacionais da rodada, com um detalhe bonito: deslizar a janela da expansão uma rodada mais cedo faz os quatro *taps* andarem junto, e a mesma fórmula que produzia `W[t]` passa a produzir `W[t+1]`. Sem estágio de pipeline novo, sem ciclo a mais — ainda são 64 rodadas por bloco.

+0,637 ns	Fase 1, sem criptografia	
-2,388 ns	com o CFS	não fecha
-1,215 ns	com esforço máximo de ferramenta	não fecha
+0,487 ns	com os dois patches de retimagem	fecha

A lição não é retimagem — é o que precisa existir antes de você poder tocar numa implementação criptográfica.

*Modificar um core de hash é a espécie de coisa que se faz errado em silêncio: o resultado continua parecendo um hash. O que tornou isto aceitável foi a rede de vetores oficiais montada antes — 65 mensagens do SHAVS, rodadas no core e através do barramento. Foi possível **provar** que a modificação não mudou o resultado, sem tocar num vetor e sem relaxar um assert.*

É a mesma razão de o FIPS 140-3 exigir self-test (seção 11) e de o projeto ter a regra “se um KAT falha, o bug está no código”. Sem isso, mexer ali seria irresponsável — e é exatamente por não ter essa rede que “não invente sua própria criptografia” é um bom conselho.

A modificação entrou como **patch versionado e revisável**, não como fork nem reescrita: `third_party/` continua byte a byte igual ao upstream, e o diff do que mudou está no repositório, legível, ao lado da justificativa.

O que falta

Vêm agora o neoTRNG e o conteúdo real da fase: os **testes de saúde** (seção 10) e o **POST** (seção 11). RCT e APT sobre a fonte bruta, e KAT de AES, SHA, HMAC e DRBG antes de aceitar qualquer comando.

Cuidado térmico registrado no plano: manter o array de osciladores em anel pequeno, meia dúzia de anéis. Centenas geram calor e ruído de alimentação localizados sem ganho de entropia.

Critérios restantes: KAT passando também no POST; 1 MB de saída do gerador passando em `ent` e `dieharder` (sanidade, não validação); e forçar falha artificial no RCT levando o dispositivo a `TAMPERED`.

28. Fase 3 — hierarquia de chaves

O coração do projeto, e onde os conceitos das Partes II e III viram código.

- **Key store** com os campos do cabeçalho TR-31 modelados desde o início — refatorar isso depois é doloroso porque o MAC cobre o cabeçalho inteiro.
- **Máquina de estados** completa, com display de 7 segmentos mostrando `Uni / Aut / OPE / tPr`.
- **Cerimônia de LMK** com três componentes XOR, KCV a cada passo, e dual control exigindo os dois botões.
- **Key blocks TR-31 versão D**: KBK e KBAK derivadas por CMAC, corpo em AES-CBC, autenticação por CMAC sobre cabeçalho e corpo.
- **Zeroize** com prova por dump.
- **Log de auditoria** gravado antes da execução, em flash, com contador monotônico.

O critério de aceitação que mais importa:

Captura da UART durante toda a suíte não contém nenhum byte de chave em claro — `grep` automatizado contra as chaves conhecidas do teste.

É a fronteira criptográfica deixando de ser afirmação e virando medida (seção 5).

Outros critérios: gerar chave, exportar em TR-31, reimportar e usar em AES com resultado idêntico; parser Python e firmware C concordando em 100 blocos aleatórios; alterar um bit do cabeçalho ou do corpo invalidando o MAC; e chave marcada como não exportável não saindo **por caminho nenhum**.

29. Fase 4 — armazenamento não-volátil

Blobs embrulhados na flash SPI, MAC de integridade, contador anti-rollback, recuperação de estado no boot.

O problema novo aqui é o **rollback**: um atacante que restaure um estado antigo da flash pode reviver uma chave revogada ou desfazer um incremento de contador. Por isso o contador monotônico precisa de proteção específica, e é um dos pontos onde HSMs reais gastam esforço desproporcional.

Detalhe de arquitetura já mapeado: a flash é a **mesma** que guarda o bitstream. Log e blobs precisam morar num offset acima da imagem de configuração, e o acesso a partir da lógica do usuário exige a primitiva `STARTUPE2` para dirigir o clock de configuração.

30. Fase 5 — API de host

Subconjunto de PKCS#11 (`C_Initialize`, `C_GenerateKey`, `C_WrapKey`, `C_Sign`) como biblioteca compartilhada, ou um command set ASCII estilo payShield.

É aqui que se entende por que a API é o que é: **é você que precisa impedi-la de vaziar material de chave**. Depois de ter lido a seção 15, escrever a API com a pergunta “o que isto vaza em laço?” na cabeça é uma experiência diferente.

31. Fase 6 — tamper e ataques

Duas metades.

Defesa: XADC monitorando temperatura e tensão, disparando zeroize fora de faixa. É o mecanismo de proteção contra falhas ambientais do FIPS nível 4 (seção 19), em versão mínima.

Ataque: atacar o próprio HSM. Glitch de clock pela porta de reconfiguração dinâmica do MMCM, observar a assinatura sair errada, e então implementar contramedidas — dupla execução, verificação de resultado (seção 17.4).

Também: bitstream encriptado com chave em BBRAM, como confidencialidade de firmware. E o experimento de RO-PUF, documentado à parte, que mede na própria placa **por que** um PUF de FPGA é ruim e por que o corretor de erros é obrigatório.

Esta é a fase mais divertida e a mais instrutiva: construir, quebrar, reforçar.

32. Fase 7 — assimétrico

RSA-2048 por Montgomery nos 90 DSP48E1; P-256 depois.

Deixado por último de propósito: o aprendizado de arquitetura de HSM está nas fases 3 a 5. E há um gancho direto com a seção 17.1 — implementar RSA-CRT sem verificar a assinatura antes de emitir é reproduzir o ataque Bellcore no próprio equipamento.

33. Ordem de trabalho, e a regra que a sustenta

Uma regra vale para todas as fases:

Nada vai para a placa sem passar antes no testbench.

Depurar criptografia por UART, no hardware, sem visibilidade interna, é a forma mais lenta possível de descobrir que faltou um byte no preenchimento. Um erro de enquadramento em simulação custa segundos; o mesmo erro na bancada custa uma tarde e várias hipóteses erradas.

A Fase 1 já demonstrou o retorno: dois defeitos reais foram pegos em simulação, e o único que escapou até o hardware foi o único que a simulação estruturalmente não podia ver.

Apêndices

A. Glossário

APT — *Adaptive Proportion Test*. Teste de saúde de fonte de entropia (SP 800-90B) que dispara quando um valor domina uma janela de amostras.

BBRAM — memória volátil do FPGA onde pode ficar a chave de decifra do bitstream. Perde conteúdo sem alimentação, e por isso é recuperável — ao contrário do eFUSE.

BDK — *Base Derivation Key*. Chave de pagamentos da qual se derivam chaves de terminal, tipicamente por DUKPT.

CFS — *Custom Functions Subsystem*. Slot do NEORV32 projetado para receber coprocessadores do usuário. É onde AES e SHA entram na Fase 2.

CMAC — código de autenticação de mensagem baseado em cifra de bloco. Usado no TR-31 para autenticar cabeçalho e corpo, e para derivar KBK/KBK.

CMVP — programa NIST/CCCS que valida módulos contra a FIPS 140-3.

CSP — *Critical Security Parameter*. Termo da FIPS para qualquer segredo cuja divulgação compromete a segurança: chave em claro, dado de autenticação, estado do gerador.

Dual control — nenhuma operação sensível pode ser realizada por uma pessoa sozinha. Ver seção 7.

DRBG — *Deterministic Random Bit Generator*. Gerador determinístico semeado por entropia física, especificado na SP 800-90A.

DUKPT — *Derived Unique Key Per Transaction*. Esquema em que cada transação usa uma chave distinta derivada de uma BDK.

eFUSE — memória de programação única do FPGA. **Proibida neste projeto:** erro vira brick permanente.

EAL — *Evaluation Assurance Level*, do Common Criteria. Mede o rigor da avaliação, **não** a força da segurança.

Fronteira criptográfica — o limite físico e lógico dentro do qual material de chave em claro existe. Ver seção 5.

KAT — *Known Answer Test*. Vetor de teste com entrada e saída conhecidas, usado nos autotestes.

KBK / KBK — chaves de cifra e de autenticação de key block, derivadas da LMK por propósito.

KCV — *Key Check Value*. Três bytes do resultado de cifrar um bloco de zeros com a chave. Permite verificar sem revelar.

KEK — *Key Encryption Key*. Chave cuja função é proteger outras chaves.

LMK — *Local Master Key*. A chave mestra do módulo, no topo da hierarquia. Nunca sai, nem embrulhada.

MMCM — *Mixed-Mode Clock Manager*. Bloco do FPGA Xilinx que sintetiza clocks. Aqui, 50 → 100 MHz.

PIN block — formato que combina PIN e número da conta para transporte. Alvo de ataques clássicos (seção 15.3).

PMP — *Physical Memory Protection*. Mecanismo RISC-V de restrição de acesso à memória por região.

POST — *Power-On Self-Test*. Bateria de KAT executada no boot, antes de aceitar comandos.

PUF — *Physically Unclonable Function*. Raiz de confiança derivada de variações de fabricação, em vez de armazenada.

RCT — *Repetition Count Test*. Teste de saúde que dispara na repetição excessiva de uma amostra.

Split knowledge — nenhuma pessoa conhece uma chave inteira; ela é montada de componentes.

Tamper-evident / resistant / responsive — a escada de proteção física. Ver seção 18.1.

TR-31 — formato de key block da indústria de pagamentos, hoje ANSI X9.143. Ver seção 8.

TRNG — gerador de números aleatórios verdadeiro, baseado em fenômeno físico.

Zeroização — destruição de material de chave de forma verificável. Ver seção 12.

B. Especificação do protocolo

Implementado em `fw/src/cmd.c` (firmware) e `host/hsmtool.py` (host).

Enquadramento

pedido	LEN(2)		CMD(1)		PAYLOAD		CRC32(4)
resposta	LEN(2)		STATUS(1)		PAYLOAD		CRC32(4)

- Todos os campos multi-byte em **big-endian**.
- `LEN` cobre `CMD` / `STATUS` + `PAYLOAD`. Não se inclui nem inclui o CRC.
- `LEN` válido: 1 a 513 (payload máximo de 512 bytes).
- **CRC32** cobre `LEN` + `CMD` / `STATUS` + `PAYLOAD` — todos os bytes menos os quatro dele próprio. Polinômio IEEE 802.3 refletido (0xEDB88320), inicialização 0xFFFFFFFF, xor final 0xFFFFFFFF. Idêntico ao `zlib.crc32` do Python.
- Timeout entre bytes no dispositivo: 250 ms. Timeout de resposta no host: 2 s.

Comandos da Fase 1

Opcode	Nome	Payload do pedido	Payload da resposta
0x01	PING	vazio	"PONG"
0x02	GET_VERSION	vazio	major, minor, patch, estado
0x03	GET_DNA	vazio	8 bytes: identidade do die, 57 bits big-endian

O GET_DNA ficou respondendo NOT_IMPLEMENTED durante toda a Fase 1 e só fechou com o CFS da Fase 2 (seção 27) — o DNA_PORT é primitiva Xilinx e precisava de um caminho até a CPU. O valor **não é segredo**: qualquer um com um cabo JTAG lê o mesmo número, e ele nunca muda. Serve para identificar a placa em log e inventário. Derivar chave dele é um erro clássico, porque público e constante são exatamente as duas propriedades que uma chave não pode ter.

Códigos de status

Código	Nome	Significado
0x00	OK	
0x01	BAD_CRC	CRC não confere
0x02	BAD_LEN	LEN fora da faixa
0x03	TIMEOUT	frame incompleto, resincronizado
0x10	UNKNOWN_CMD	opcode não está na tabela
0x11	BAD_PARAM	tamanho ou conteúdo do payload
0x12	NOT_IMPLEMENTED	na tabela, sem implementação ainda
0x20	WRONG_STATE	comando não permitido neste estado
0x21	NOT_AUTHORIZED	falta dual control
0x22	NOT_EXPORTABLE	exportabilidade do slot proíbe
0x30	SELFTTEST_FAIL	
0x31	TAMPERED	
0xFF	INTERNAL_ERROR	

Exemplo verificado em hardware

```
pedido   : 00 01 01 91 5D D8 C5
resposta : 00 05 00 50 4F 4E 47 FB 28 3D 2A
          ^^ P O N G
```

91 5D D8 C5 é `zlib.crc32(b'\x00\x01\x01')`.

C. Pinagem

Fonte: esquemáticos QMTECH e verificação em hardware. Detalhes e procedência em `doc/pinout.md`.

Função	Pino	Estado
Clock 50 MHz	N11	verificado em hardware
Reset (SW1)	B7	esquemático; ativo baixo
UART RX / TX	T15 / T14	verificado em hardware
Dual control A (SW2)	M6	esquemático; ativo baixo
Dual control B (SW5)	P6	esquemático; ativo baixo
LEDs D1–D5	R6, T5, R7, T7, R8	D1 e D2 verificados; ativos baixos
7 segmentos	T10, K13, P11, R11, R10, N9, K12, P9	pinos oficiais
Dígitos (varredura)	T9, P10, T8	3 dígitos

Armadilha registrada: com o gravador ligado existem **duas** portas seriais. O adaptador JTAG usado é um FT232H que também cria `/dev/ttyUSB*`, e costuma enumerar **antes** da placa. Escolher a porta por ordem alfabética acerta o cabo de gravação, e o sintoma é um timeout sem explicação. A ferramenta de host escolhe por identificador USB por causa disso.

Pendências: cores dos LEDs (a Fase 3 precisa saber se D5 é vermelho para o `TAMPERED`), polaridade do display de 7 segmentos, e a ordem física dos botões no silk.

D. Reproduzir o trabalho

Dependências

```
git clone --recurse-submodules <url>

sudo apt install -y gcc-riscv64-unknown-elf picolibc-riscv64-unknown-elf
sudo apt install -y python3-serial openfpgaloader
```

Vivado 2026.1 (o `xsim` também é usado na simulação, por causa das primitivas Xilinx e do VHDL do NEORV32).

O compilador sozinho não basta: ele fornece apenas os headers freestanding, e o header do NEORV32 precisa de uma biblioteca C. Não há `libnewlib` para RISC-V nos repositórios Debian; `picolibc` é o equivalente.

Construir e testar

```
make -C fw image           # firmware; o build da FPGA depende dele
source /opt/AMD/2026.1/Vivado/settings64.sh

./scripts/sim.sh           # todos os testbenches
vivado -mode batch -source scripts/build.tcl

./scripts/program.sh       # grava na RAM de configuração (volátil)

python3 host/hsmtool.py selftest # sem placa
python3 host/test_hsmtool.py     # sem placa
python3 host/hsmtool.py ping
python3 host/hsmtool.py bench -n 10000
```

Política de código de terceiros

Nunca editar `third_party/` diretamente: a mudança não fica guardada no repositório, é descartada por `git submodule update`, e quebra a procedência — o pin continua dizendo `v1.13.3` enquanto o bitstream contém outra coisa.

Ordem para mudar código externo:

1. **Configuração** (generic no wrapper) — resolve quase sempre
2. **Ponto de extensão projetado** — é o caso do CFS na Fase 2
3. **Patch** em `patches/`, aplicado pelo build
4. **Fork** e repontar o submódulo — só com patch que precise persistir

Detalhes em `doc/submodulos.md`. O espelho offline reproduzível dos cores externos sai de `./scripts/mirror-deps.sh`.

E. Leitura

Normas

- **FIPS 140-3** e ISO/IEC 19790 — requisitos de módulo criptográfico
- **NIST SP 800-90A/B/C** — DRBG, fontes de entropia, construções
- **ANSI X9.143** (sucessor do TR-31) — key blocks
- **PCI PIN Security Requirements** e **PCI PTS HSM** — de onde vêm dual control e split knowledge como exigência

Ataques

- Bond, M. — *Attacks on Cryptoprocessor Transaction Sets* (2001)
- Bond, M. e Zieliński, P. — *Decimalisation Table Attacks for PIN Cracking*

2003.

- Clulow, J. — *On the Security of Real-World PIN Processing* (2003)
- Boneh, DeMillo e Lipton — *On the Importance of Checking Cryptographic Protocols for Faults* (1997) — o ataque Bellcore

- Kocher, P. — *Timing Attacks* (1996) e, com Jaffe e Jun, *Differential Power Analysis* (1999)
- Piret e Quisquater — análise diferencial de falhas em AES (2003)

Livros

- Anderson, R. — *Security Engineering*. Os capítulos sobre APIs de segurança e sobre segurança de hardware são a melhor introdução ao assunto da Parte III.
- Maes, R. — *Physically Unclonable Functions*. Para o experimento da Fase 6.

Ferramentas e cores usados

- **NEORV32** — processador RISC-V em VHDL, github.com/stnolting/neorv32
- **secworks/aes** e **secworks/sha256** — cores criptográficos para a Fase 2
- **openFPGALoader** — gravação por JTAG

Este documento acompanha o repositório `hsm-a7`. A Parte V reflete o estado ao fim da Fase 1, validada em hardware. As Partes I a IV são independentes do projeto e valem por si.